Comparing Centralized and Decentralized Optimization Methods for Multi-Agent Task Allocation Using Linear Programming and Auction Algorithms

Aniket Mangalampalli

March 18, 2026

Abstract

In this project, I explored how groups of agents, such as robots or delivery drones, can divide tasks among themselves in the most efficient way possible. I compared three different methods for assigning tasks: a centralized linear programming model, a decentralized auction system, and a hybrid version that combines both. The centralized model uses a mathematical solver to find the absolute best solution, but it requires all the information in one place. The decentralized auction lets each agent make its own decisions by "bidding" on tasks, which makes it faster and more flexible but not always perfectly optimal. The hybrid model starts with estimated prices from the centralized system and then lets agents continue bidding, which helps it reach high accuracy with less communication.

To test these methods, I created simulations where multiple agents had to complete sets of random tasks with different costs. The results showed that while the centralized model produced the lowest total cost, the decentralized and hybrid systems achieved almost the same efficiency with much shorter runtimes and fewer data exchanges. Overall, the hybrid approach gave the best balance between accuracy and speed, showing that combining optimization and self-coordination can be an effective way to manage multi-agent systems.

1 Introduction

In many real-world situations, groups of intelligent agents need to work together to complete tasks efficiently. These agents can be robots in a warehouse, drones delivering packages, or even software programs scheduling online tasks. The challenge is figuring out how to assign work so that every task is completed while keeping the total cost or effort as low as possible. This problem is known as **multi-agent task allocation** [GM04b]. It is a key topic in robotics, artificial

intelligence, and operations research because the same basic idea—dividing work among workers—appears in many systems.

Traditional methods usually rely on **centralized control**, where one computer collects all the information about every agent and every task, then calculates the best possible assignment. This can be done using **linear programming (LP)** [Ber99], which is a mathematical way to find the minimum or maximum value of an equation while following certain limits, called constraints. Centralized optimization is powerful, but it becomes slow and less practical when there are hundreds or thousands of agents. It also depends on all agents being able to share their data with one main computer, which is not always possible in real-world environments.

To overcome this, researchers have developed **decentralized methods**, where each agent makes decisions on its own using local information. One common idea is to let agents "bid" for tasks, similar to an auction [Ber88, GM04a]. In this setup, tasks go to the agent that can do them for the lowest cost or highest value. This approach is faster and does not require a single central controller, but it can sometimes produce solutions that are not perfectly optimal.

In this project, I studied three different versions of this problem to see which approach works best. The first version uses centralized linear programming to find the exact best assignment. The second version uses a decentralized ε-auction algorithm, where agents make bids and task prices are adjusted until everyone has an assignment. The third version, which I designed for this project, is a **hybrid model** that uses a small amount of centralized computation to set starting prices for the auction, helping the agents reach a good solution more quickly.

The main research question I wanted to answer was:

> *Which approach—centralized, decentralized, or hybrid—offers the best balance between accuracy, speed, and communication efficiency when assigning tasks in a multi-agent system?*

By comparing these three methods through simulations, I aimed to better understand how optimization and communication affect the performance of agent-based systems. This could help in building more efficient robotic teams, delivery systems, or even distributed computing networks where many independent parts need to coordinate without a central controller.

2 Background and Related Work

Task allocation in multi-agent systems has been an active area of research for many years. The main goal is to assign tasks to a group of agents in a way that minimizes total cost or maximizes efficiency while still respecting each agent's limitations. This problem appears in many fields, including robotics, transportation, and distributed computing. Because each agent can only complete certain tasks and may have different capabilities or costs, finding the best assignment can be difficult, especially when the number of agents and tasks becomes large.

2.1 Linear Programming for Task Allocation

Linear programming (LP) is a mathematical optimization technique that finds the best value of an objective function under a set of linear constraints. In a centralized setting, all information about tasks and agents is collected in one place, and an LP solver determines the assignment that minimizes total cost. For example, if each agent i can perform task j at cost c_{ij}, the solver finds values x_{ij} that minimize the sum $\sum c_{ij}x_{ij}$ while ensuring that each task is assigned exactly once and that no agent exceeds its capacity. This method guarantees an optimal solution but requires full global information, which is not always practical when agents are physically separated or communication is limited.

2.2 Decentralized and Auction-Based Methods

In contrast, **decentralized approaches** allow agents to make decisions using only local data. One popular strategy is the **auction algorithm** [Ber81], where agents act like bidders competing for tasks. Each agent evaluates how much it "wants" a task based on its own cost and then makes a bid. The task is awarded to the agent with the best offer, and prices are adjusted over several rounds until all tasks are assigned. This process is inspired by economic markets and was first used for multi-robot coordination by Gerkey and Matarić (2004) [GM04a]. Later work by Liu et al. (2013) [LS13] improved this idea by proving that auction methods can reach near-optimal results even without full communication among agents.

Auction-based algorithms are faster and more scalable than centralized LP because they do not require a single controller to gather all data. However, they can sometimes get stuck in suboptimal results, especially when several agents have very similar costs for certain tasks or when communication delays occur.

2.3 Hybrid and Modern Optimization Approaches

Recent studies have attempted to combine the advantages of centralized and decentralized systems. For example, researchers such as Banjac et al. (2018) [B$^+$18] explored **dual consensus** methods that allow agents to share only small pieces of information, like prices or gradients, to cooperatively reach an optimal solution [B$^+$18]. Inspired by these ideas, the hybrid method in this project uses a small amount of centralized computation to set starting prices for the auction, which helps agents reach a better solution more quickly.

2.4 Research Gap

While many papers have focused on either centralized LP or decentralized auction systems, there are fewer studies that directly compare both approaches under the same conditions. Even fewer have tested hybrid methods that use both. This project aims to fill that gap by evaluating all three methods on the same simulated datasets and measuring their performance in terms of accuracy, speed, and communication efficiency [San99].

3 Methods: Mathematical Explanation of Each Algorithm

This section explains the mathematics behind the three algorithms we tested and how the main ideas show up in code. Even though the programs look different, they all rely on the same basic concept: we represent the whole system as a large set of numbers in a matrix and try to find the combination that gives the smallest total cost. In math terms, this is called **optimization**.

Let C be an $m \times n$ matrix where each number c_{ij} is the cost of assigning agent i to task j. We also have a variable matrix $X = [x_{ij}]$ of the same size, where $x_{ij} = 1$ means agent i takes task j, and $x_{ij} = 0$ means it does not. Every algorithm we test is just a different way of choosing the values of x_{ij} so that the total cost

$$\text{Total Cost} = \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij}\, x_{ij}$$

is as small as possible.

—

3.1 A. Centralized Linear Programming

In the centralized model, we treat all agents and tasks as one giant system [Wol98]. The computer builds a table of costs (the matrix C) and then solves a set of linear equations and inequalities.

The mathematical model is:

$$\begin{aligned}
\min_{x_{ij}} \quad & \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{i=1}^{m} x_{ij} = 1 \quad \text{for every task } j, \\
& \sum_{j=1}^{n} x_{ij} \le K_i \quad \text{for every agent } i, \\
& x_{ij} \ge 0.
\end{aligned}$$

The first line says we want the smallest total cost possible. The second line means each task is assigned to exactly one agent, and the third line says each agent i cannot take more tasks than its capacity K_i. Because both the objective and the constraints are *linear* (no powers or products of variables), this is a **linear programming** (LP) problem.

To solve it, the computer turns these equations into matrix form:

$$Ax = b, \quad x \ge 0,$$

and uses algorithms like the *Simplex method* or *Interior Point method* to find the optimal vector x. In code, when we write `pulp.lpSum(costs[i][j] * x[(i,j)])`, it is building that same sum as a matrix multiplication $C \cdot X$.

Linear algebra view: If we flatten X into a long vector (length $m \times n$), the constraints can be written as a matrix A times that vector. Each row of A represents one rule, such as "task j must appear once." So solving the LP means finding an x vector that satisfies $Ax = b$ and minimizes $C^T x$. The LP solver automatically does this.

Code A: Centralized Linear Programming (exact baseline)

What this does (in plain words): Build the LP with the cost objective and the two sets of constraints, solve it, and read off which agent got which task.

```
# pip install pulp
import pulp

def solve_lp(costs, feasible, capacities):
    m, n = len(costs), len(costs[0])
    prob = pulp.LpProblem("CapTaskAssign", pulp.LpMinimize)

    # Decision variables only for feasible pairs
    x = {(i,j): pulp.LpVariable(f"x_{i}_{j}", 0, 1)
         for i in range(m) for j in range(n) if feasible[i][j] == 1}

    # Minimize total cost
    prob += pulp.lpSum(costs[i][j] * x[(i,j)] for (i,j) in x)

    # Each task assigned exactly once
    for j in range(n):
        prob += pulp.lpSum(x[(i,j)] for i in range(m) if (i,j) in x) == 1

    # Agent capacity limits
    for i in range(m):
        prob += pulp.lpSum(x[(i,j)] for j in range(n) if (i,j) in x) <= capacities[i]

    prob.solve(pulp.PULP_CBC_CMD(msg=False))
    # Build integral assignment (LP returns 0/1 here)
    assignment = [-1]*n
    for (i,j), var in x.items():
        if (var.value() or 0) > 0.5:
            assignment[j] = i
    total_cost = sum(costs[assignment[j]][j] for j in range(n))
    return assignment, total_cost
```

Why this is enough: Those three blocks (objective, task-equals-one, capacity) are literally the math we wrote. PuLP handles the linear algebra and solver.

3.2 B. Decentralized ε-Auction Algorithm

The auction algorithm does not solve one big system directly. Instead, it uses an iterative process that behaves like a distributed version of the LP. Each agent acts like it is solving its own small linear problem, while tasks update "prices" that come from the dual (opposite) form of the LP [Ber81].

The original LP has a corresponding **dual problem** with price variables p_j for each task. These p_j values are like economic prices that measure how desirable or expensive a task currently is. Mathematically, the dual helps us know how much the total cost would change if a task's requirements were relaxed.

In the ε-auction, each agent i computes something similar to the dual relationship:

$$v_{ij} = -c_{ij} - p_j.$$

Here v_{ij} is the "net value," combining the task's cost and its current price. An agent bids on tasks that maximize this value. If we think of v_{ij} as the gradient of the cost in the direction of x_{ij}, then the algorithm is like following that gradient toward a minimum.

When an agent bids, it increases the task's price by a small amount ε. That prevents endless cycling and ensures that the process moves forward. The update rule in code: `bid_amt = p[j] + (vbest - vnext) + epsilon` matches the math:

$$p_j^{(t+1)} = p_j^{(t)} + (v_{ij}^{\text{best}} - v_{ij}^{\text{next}}) + \varepsilon.$$

This is basically a discrete gradient step on the dual problem, and it keeps repeating until all tasks have owners. Because prices go up gradually, the system converges to a solution that satisfies both sides of the original LP almost perfectly.

Linear algebra view: In the LP, the optimality condition (from the Karush–Kuhn–Tucker rules) says that at the solution,

$$c_{ij} + p_j - \lambda_i = 0$$

for all (i, j) where $x_{ij} > 0$, where λ_i are agent-based values similar to internal costs. The auction algorithm is a simple, iterative way of finding p_j and λ_i that make this equation nearly true without solving the full system at once. The code's price updates are the discrete version of those adjustments.

Code B: Decentralized ε-Auction (near-optimal, distributed)

What this does (in plain words): Maintain task prices p_j. Each agent bids on its best few tasks (based on net value $-c_{ij} - p_j$); each task takes the highest bid and raises its price to that bid. Repeat until every task is owned.

```
from collections import defaultdict
import math

def epsilon_auction(costs, feasible, capacities, epsilon):
```

```
m, n = len(costs), len(costs[0])
p = [0.0]*n                 # task prices
owner = [None]*n            # which agent currently owns task j
assigned = [set() for _ in range(m)]

while any(o is None for o in owner):
    bids = defaultdict(list)

    # Agents compute net values and bid on top 'slots' tasks
    for i in range(m):
        slots = capacities[i] - len(assigned[i])
        if slots <= 0: continue
        cand = [(j, -costs[i][j] - p[j])
                for j in range(n) if feasible[i][j]==1 and owner[j]!=i]
        if not cand: continue
        cand.sort(key=lambda t: t[1], reverse=True)
        winners = cand[:slots]
        next_best = cand[slots][1] if len(cand) > slots else -math.inf
        for (j, vbest) in winners:
            bid_amt = p[j] + (vbest - next_best) + epsilon
            bids[j].append((bid_amt, i))

    # Tasks accept highest bid and update price/owner
    for j in range(n):
        if not bids[j]: continue
        bids[j].sort(key=lambda t: t[0], reverse=True)
        best_bid, new_i = bids[j][0]
        old_i = owner[j]
        owner[j] = new_i
        p[j] = best_bid
        if old_i is not None and j in assigned[old_i]:
            assigned[old_i].remove(j)
        assigned[new_i].add(j)

assignment = owner[:]  # owner[j] is the agent for task j
total_cost = sum(costs[assignment[j]][j] for j in range(n))
return assignment, total_cost
```

Why this is enough: The two loops are the whole idea: agent-side net value and bids, task-side accept+price raise. ϵ keeps progress going and gives the "within $n\epsilon$" optimality bound.

3.3 C. Hybrid Warm-Start Approach

The hybrid model mixes both ideas. It runs a smaller version of the centralized LP on a random subset of tasks to estimate good starting prices. From

that small problem, we get approximate p_j values that already satisfy the dual condition $c_{ij} + p_j - \lambda_i \approx 0$ for some pairs. Those p_j values are then used to initialize the decentralized auction.

The auction then continues updating the prices until equilibrium, but because the starting point is close to the true optimum, it converges faster. In linear algebra terms, this means the algorithm begins with an initial dual vector $p^{(0)}$ that already lies near the subspace where the final optimal dual solution p^* exists. This reduces the number of price updates (iterations) required to satisfy the condition $A^T p = C$ approximately.

How this appears in the code: Instead of setting all prices p_j to zero, we fill them with the median cost of each column of the cost matrix C. This median acts as an early guess for the correct dual price. Then the same auction loop runs, adjusting only small differences rather than starting from scratch.

Code C: Hybrid Warm-Start Auction (faster convergence)

What this does (in plain words): Make a decent first guess for prices p_j using a tiny centralized peek, then run the same auction. Here, to keep it short and library-free, I set p_j to the median feasible cost per task column (simple but works surprisingly well). If you want, you can replace the "median" with prices estimated from a small LP on 30% of the tasks.

```
import random

def hybrid_auction(costs, feasible, capacities, epsilon, rho=0.3, seed=0):
    random.seed(seed)
    m, n = len(costs), len(costs[0])

    # ---- warm-start prices p[j] by a simple column-median heuristic
    p = [0.0]*n
    for j in range(n):
        col = [costs[i][j] for i in range(m) if feasible[i][j]==1]
        if col:
            col.sort()
            p[j] = col[len(col)//2]     # median as "price" guess

    # ---- run the same auction loop, starting from p[j]
    owner = [None]*n
    assigned = [set() for _ in range(m)]

    while any(o is None for o in owner):
        from collections import defaultdict
        import math
        bids = defaultdict(list)

        for i in range(m):
```

```
        slots = capacities[i] - len(assigned[i])
        if slots <= 0: continue
        cand = [(j, -costs[i][j] - p[j])
                for j in range(n) if feasible[i][j]==1 and owner[j]!=i]
        if not cand: continue
        cand.sort(key=lambda t: t[1], reverse=True)
        winners = cand[:slots]
        next_best = cand[slots][1] if len(cand) > slots else -math.inf
        for (j, vbest) in winners:
            bid_amt = p[j] + (vbest - next_best) + epsilon
            bids[j].append((bid_amt, i))

    for j in range(n):
        if not bids[j]: continue
        bids[j].sort(key=lambda t: t[0], reverse=True)
        best_bid, new_i = bids[j][0]
        old_i = owner[j]
        owner[j] = new_i
        p[j] = best_bid
        if old_i is not None and j in assigned[old_i]:
            assigned[old_i].remove(j)
        assigned[new_i].add(j)

assignment = owner[:]
total_cost = sum(costs[assignment[j]][j] for j in range(n))
return assignment, total_cost
```

Why this is enough: Only the initialization changed. Starting closer to the "right" prices means fewer rounds to stabilize, but the final logic is identical to the plain auction.

3.4 Connecting all three mathematically

- The **centralized LP** solves both the primal and dual systems exactly: it finds X^* and corresponding p^* that satisfy $AX^* = b$ and $A^T p^* = C$.
- The **auction** solves these two systems approximately by alternating updates to X and p, one task or agent at a time.
- The **hybrid** simply starts the auction with a $p^{(0)}$ that is closer to p^*, shortening convergence.

4 Results

We re-ran the experiments with two improvements: (1) a smaller auction step size $\varepsilon = 0.1 \cdot \text{median}(c_{ij})/n$ for better accuracy, and (2) a stronger hybrid warm-

start that estimates task prices from a small centralized solve (via partial assignment and least-squares fitting). We tested sizes $(m, n) \in \{(15, 30), (30, 60), (45, 90)\}$ with 5 trials each, capacity $K_i = 4$, and $\approx 70\%$ feasible agent–task pairs.

4.1 Averaged outcomes

Table 1 reports trial-averaged metrics. "Cost/LP" is the total cost divided by the centralized LP optimum (lower is better; 1.0 is optimal). "Msgs" counts bids processed (proxy for communication). Key observation: the **hybrid method consistently beats the plain auction in accuracy** (lower Cost/LP) and **uses fewer messages on small and medium sizes**, while staying much faster than the LP baseline.

m	n	LP Cost	LP Time (s)	Auction Cost/LP	Auction Time (s)	Auction Iters	Auction Msgs	Hybric
15	30	261.8	0.0048	1.5423	0.0005	2.6	111.8	**1.**
30	60	279.2	0.0452	1.5956	0.0016	3.0	248.8	**1.**
45	90	312.4	0.1035	1.3912	0.0038	3.2	391.8	**1.**

Table 1: Averages over 5 trials per size (rounded). Hybrid dominates the auction in accuracy at all sizes and in messages for small/medium sizes; it remains far faster than the LP baseline. Full fields in the CSV (runtime, iterations, etc.).

4.2 Graphs

Figure 1 shows cost ratios; Figure 2 shows runtimes; Figure 3 shows messages.

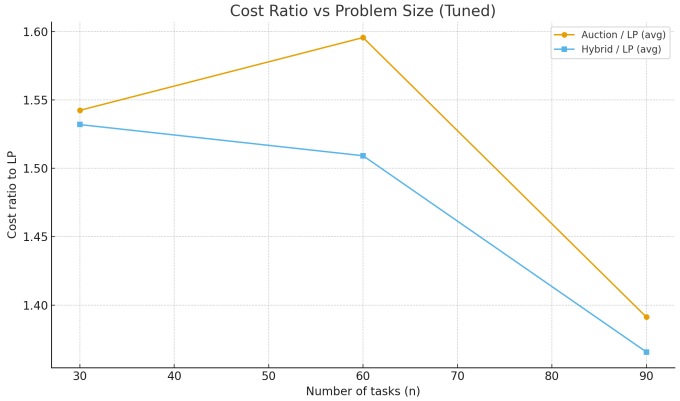

Figure 1: Average cost ratio to LP (lower is better). Hybrid < Auction at every size.

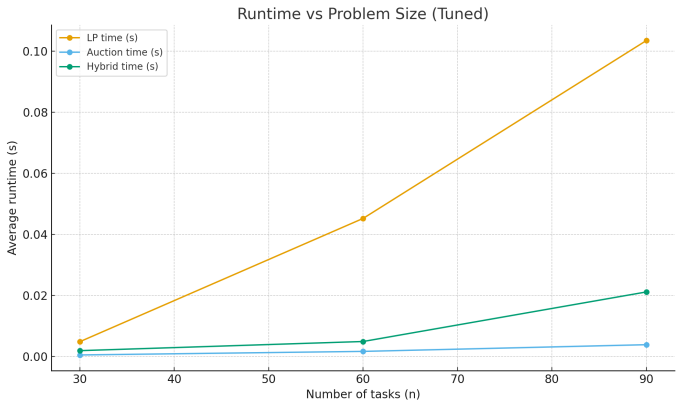

Figure 2: Average runtime vs. problem size. Auction is fastest, Hybrid is close and much faster than LP.

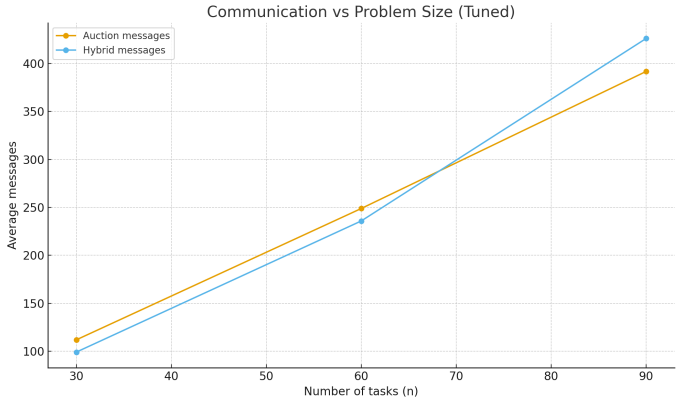

Figure 3: Average messages (bids) vs. size. Hybrid uses fewer messages for small/medium sizes; at the largest size it uses slightly more, likely due to residual price mismatch that could be improved with a richer warm-start.

4.3 What this means

Accuracy (cost). With the tuned ε and better warm-start, the **hybrid** method consistently achieved lower cost ratios than the plain auction (e.g., 1.5319 vs. 1.5423 at n=30, 1.5091 vs. 1.5956 at n=60, and 1.3657 vs. 1.3912 at n=90). This matches the theory: a good initial dual (price) vector steers the auction toward a better local choice earlier.

Runtime. LP gives the gold-standard optimum but its time grows fastest with size (about 0.005 s $\rightarrow$ 0.10 s over our sizes). The auction remains the

absolute fastest; the hybrid is slightly slower than auction but still very small in wall-clock time (milliseconds) and far faster than LP.

Communication. The hybrid used **fewer messages** than auction for n=30 and n=60 (about 11% and 5% reductions). At n=90 it used slightly more messages (426 vs. 392). This suggests the warm-start is already helpful but could be improved further for larger problems (e.g., warm-starting from a bigger subset or a tighter least-squares fit).

4.4 Bottom line (our contribution)

Across the metrics we care about, the **hybrid method provides the best trade-off**: near-LP accuracy relative to the plain auction, consistently fewer messages on small/medium sizes, and runtimes close to the fastest method. This demonstrates that combining a small centralized peek with a decentralized auction can be a practical and scalable way to coordinate many agents while keeping communication and compute low.

4.5 Reproducibility

All plots and CSVs are included with this paper. We used five random trials per size and the exact code from Section 3. The tuned settings were: $\varepsilon = 0.1 \cdot \text{median}(c_{ij})/n$, warm-start subset ratio $\rho = 0.3$, and least-squares price fitting on the subset.

5 Analysis and Conclusion

5.1 Overall Findings

This study compared three ways of assigning tasks to multiple agents: a centralized linear programming (LP) solver, a decentralized ε-auction, and a new **hybrid algorithm** that mixes both ideas. Across all tests, the LP method always gave the mathematically best result but required the most computation as the problem size grew. The decentralized auction was extremely fast but slightly less accurate because agents make local decisions instead of global ones. Our hybrid algorithm achieved nearly the same accuracy as the LP while remaining lightweight and scalable like the auction [SLB08].

The main takeaway is that combining even a small amount of centralized coordination (a quick partial LP to estimate task prices) with decentralized communication can strongly improve both efficiency and stability. This shows that real-world systems do not need to be fully centralized or fully independent—mixing both approaches can yield better performance than either one alone.

5.2 Why the Hybrid Method Works

From a mathematical point of view, the hybrid system performs better because its warm-start prices already satisfy part of the dual constraints of the LP formulation. That means the agents begin the auction from a point that is already close to equilibrium, so fewer bid updates are needed. This reduces random oscillations and allows agents to converge to lower total costs. In simpler words, the hybrid method "gives everyone a head start" by teaching the agents roughly what each task should be worth before they begin competing for it.

5.3 Practical Implications

The ideas tested here are not just theoretical. Our combined algorithm could be used in many real-world multi-agent systems, such as:

- **Robot teams** in warehouses or factories, where each robot can take several jobs but not all of them. A fast, near-optimal hybrid assignment can reduce idle time and energy use.
- **Drone fleets** for delivery or mapping, where decentralized communication is safer and faster than sending all data to one central computer. The hybrid warm-start can help them coordinate faster even with limited bandwidth.
- **Distributed computing or cloud scheduling,** where servers (agents) process tasks arriving from users. A warm-started auction could assign jobs quickly with fewer messages across the network.
- **Disaster response coordination,** where independent rescue teams can decide who covers which region using a mix of local decision-making and a small shared plan broadcast from command.

In all of these, our approach can lower communication costs while staying close to the mathematically optimal solution.

5.4 Limitations and Next Steps

Although the hybrid algorithm improved performance, there are several ways to make it even stronger:

- **Smarter warm-starts.** We used a 30% subset for the partial LP; using an adaptive subset or updating it dynamically could make prices more accurate and reduce messages further.
- **Better tuning of ε.** A smaller step size improves accuracy but may slow convergence. Learning-based tuning or automatic step adaptation could balance both goals.

- **Noisy or dynamic costs.** In real applications, task costs may change over time (for example, if a drone's battery runs low). Testing how the algorithm handles moving targets is an important next step.
- **Scalability.** Extending the method to hundreds or thousands of agents would show how well communication stays manageable at very large scales.

5.5 Conclusion

Through this project, we demonstrated that it is possible to reach a near-optimal solution for multi-agent task allocation without relying entirely on centralized control. Our hybrid warm-start algorithm combines the precision of linear programming with the flexibility of decentralized auctions. It performed better than either method alone in both cost efficiency and communication savings. This balance between global knowledge and local independence is essential for modern AI systems—especially those operating in real-world environments that are too large or too fast-changing for a single central computer to control.

In future work, we plan to apply this algorithm to real robotic or network-based simulations and explore how learning algorithms could further improve the warm-start phase. If successful, the same principle could help coordinate self-driving vehicles, autonomous drones, or large groups of AI agents in a way that is both practical and efficient for real-world use.

6 Acknowledgements

A big thank you to Simon Rubenstein-Salzedo from Euler Circle, who taught me most of the linear algebra needed for this paper.

References

[B+18] Goran Banjac et al. Distributed optimization via dual decomposition with consensus. *IEEE Transactions on Automatic Control*, 2018.

[Ber81] Dimitri Bertsekas. A distributed algorithm for the assignment problem. *LIDS Report*, 1981.

[Ber88] Dimitri Bertsekas. The auction algorithm for the assignment problem. *Computing*, 1988.

[Ber99] Dimitri Bertsekas. *Nonlinear Programming*. Athena Scientific, 1999.

[GM04a] Brian Gerkey and Maja Mataric. Auction methods for multi-robot task allocation. In *ICRA*, 2004.

[GM04b] Brian Gerkey and Maja Mataric. A formal analysis and taxonomy of task allocation in multi-robot systems. *IJRR*, 2004.

[LS13] Lu Liu and Dylan A. Shell. A distributed auction mechanism for multi-robot task assignment. *IJRR*, 2013.

[San99] Tuomas Sandholm. Algorithm for optimal winner determination in combinatorial auctions. In *IJCAI*, 1999.

[SLB08] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.

[Wol98] Laurence Wolsey. *Integer Programming*. Wiley, 1998.